Web Scraping: Data Gathering from Web

Eunji Kim
eunjikim@cau.ac.kr

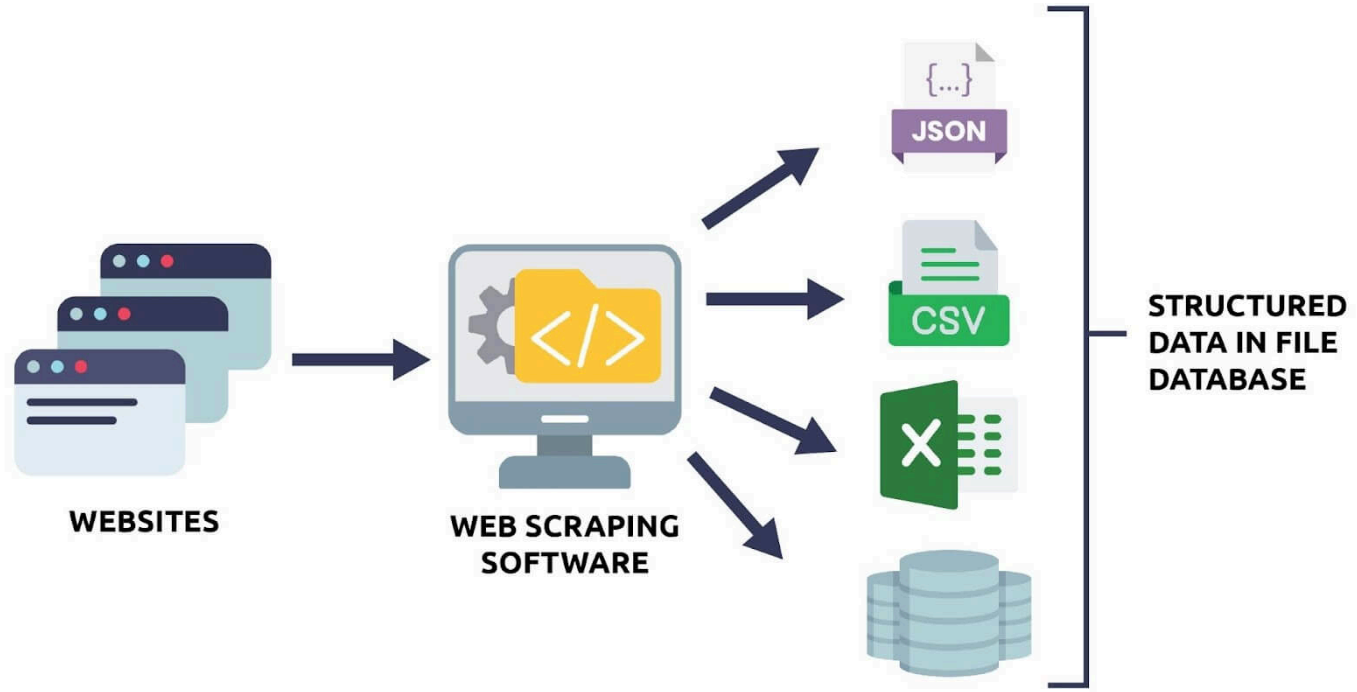
Outline

- What is Web Scraping?
- What is HTML?
- Preparation for Web Scraping: Selenium and Chrome Driver
- Web Scraping

What is Web Scraping?

Web Scraping

- Process of *automatically* extracting specific data from a target website
- Usually done with a specific purpose of data analysis



Example: Develop a Price Tracking System

Web scraping is necessary to track the prices of the target products:

- Target website: Coupang
- Target data: Daily min/max prices

Applications of Web Scraping

Businesses leverage web scraping tools to extract data from websites.

1. **E-commerce price tracking**: Scrape product names, prices, and ratings from sites like Amazon or eBay to monitor price changes over time.
2. **News monitoring**: Scrape headlines, article text, and publication dates from news websites to perform sentiment analysis or track trending topics.
3. **Social media analysis**: Extract posts, hashtags, likes, and comments from platforms like Twitter or Instagram for research or marketing insights.
4. **Market analysis**: Collect product prices, customer reviews, and ratings from online stores to analyze pricing trends, customer preferences, and competitor positioning.
5. **Stock price prediction**: Extract prices of target stocks from websites to develop and validate a stock price prediction model

Key steps typically include:

1. **Identify the target website** – Determine which website and which pages contain the data you need.
2. **Inspect the web page structure** – Use browser developer tools to understand the HTML elements where the data resides.
3. **Send requests to the website** – Programmatically fetch the web page content (often using HTTP requests).
4. **Parse the data** – Extract the relevant information from the HTML, JSON, or other formats (using tools like BeautifulSoup, lxml, or Selenium).
5. **Store the data** – Save the extracted data in a structured format like CSV, JSON, or a database.
6. **Respect legal and ethical considerations** – Check the website's terms of service, robots.txt, and avoid overloading the server.

What is HTML?

HTML (HyperText Markup Language)

- Web pages have many visual and textual elements.
- HTML codes serve to position all of this content on web pages.

```
<!DOCTYPE html>
    <html>

        <head>
            <title> Title here </title>
        </head>

        <body>
            Web page content goes here.
        </body>

    </html>
```

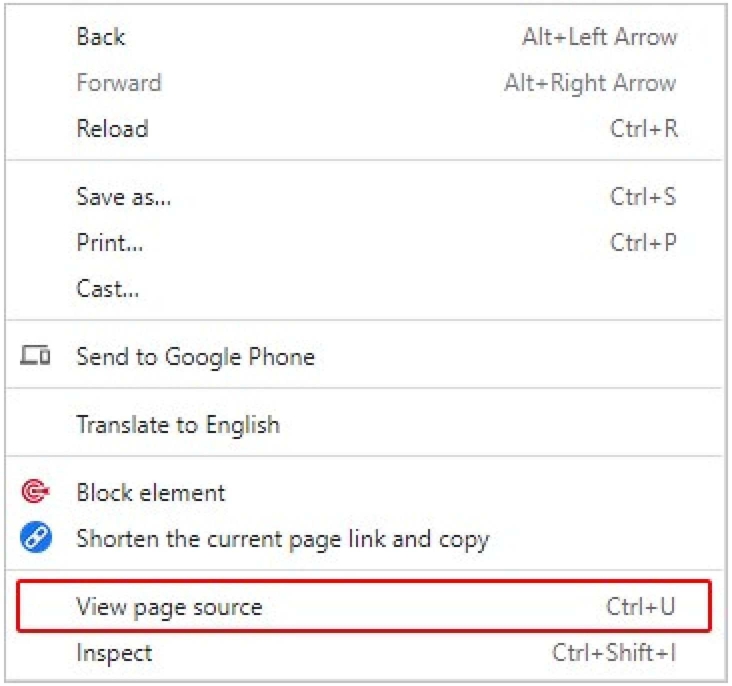
Example page

Daum Sports News: https://sports.daum.net/worldsoccer/news/breaking

How to check HTML source code

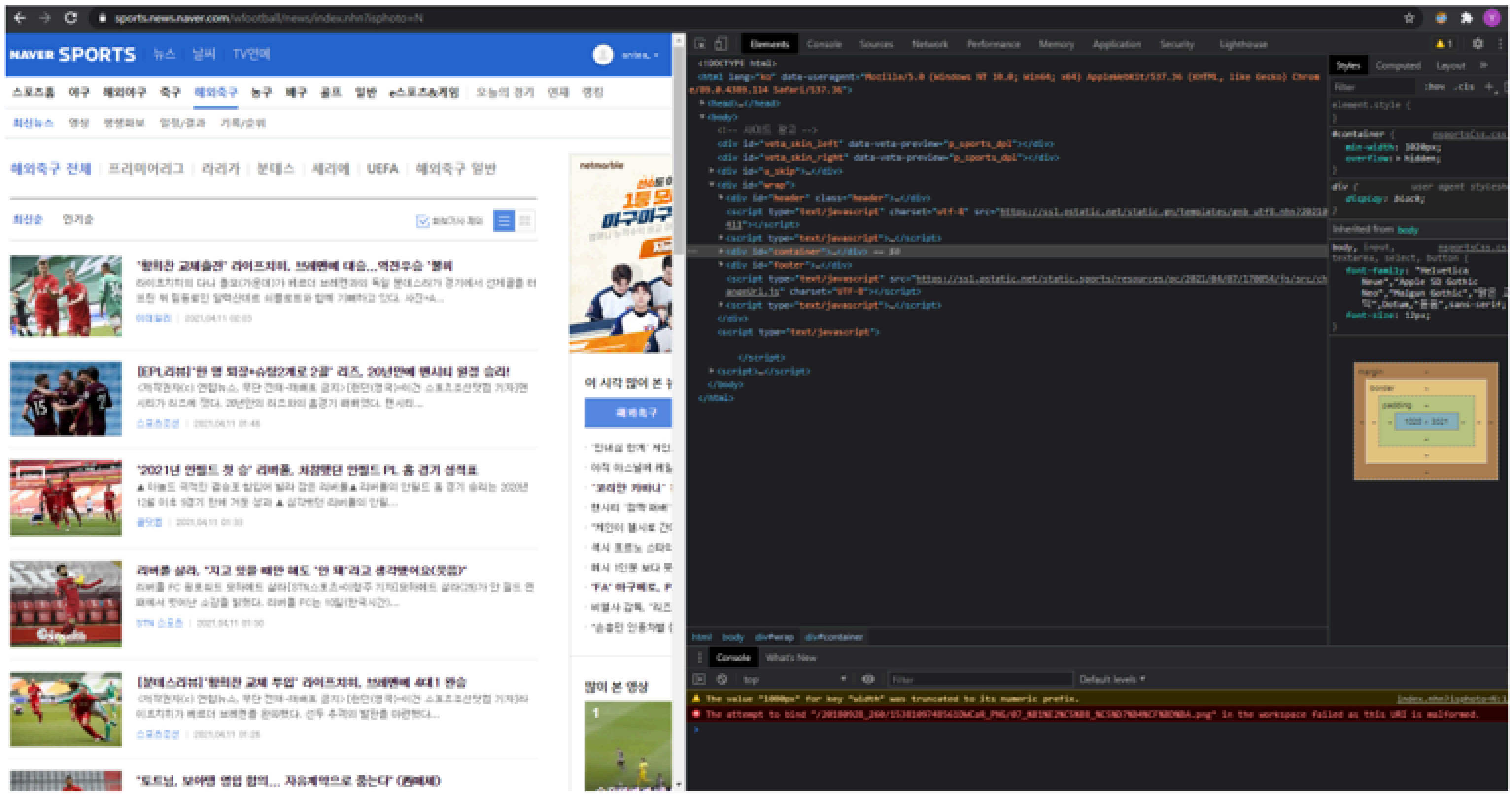
To see the HTML code of the webpage you are currently viewing,

- Place mouse cursor on an empty space on the page and right-click
- Select "View page source (페이지 소스 보기)" (Shortcut: `Ctrl` + `U`)



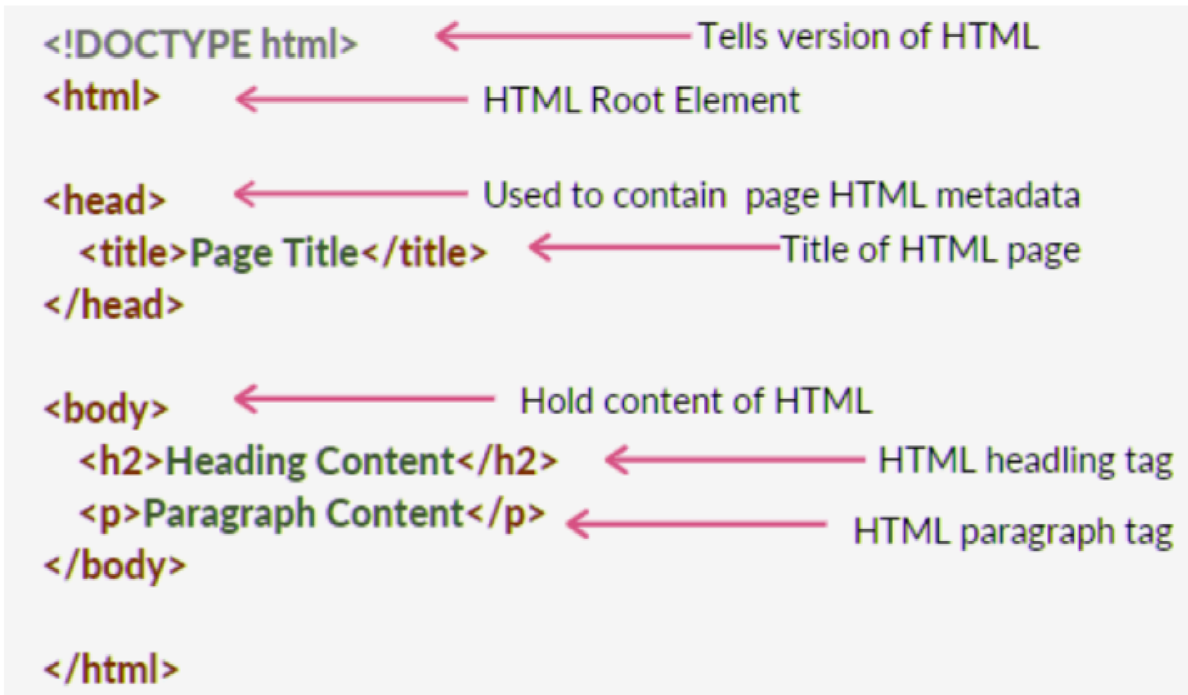
Or, you can use the **Inspect (검사)** menu for more functions:

- Place mouse cursor on an empty space on the page and right-click
- Select "Inspect (검사)" (Shortcut: `Ctrl` + `Shift` + `I`)



HTML Page Structure

It contains the essential two main building-blocks, `head` and `body`, upon which all web pages are created.



HTML Tag

- The actual keyword or name enclosed in angle brackets (`< >`)
- Tells the browser what kind of content to expect
- Example: `<a>` tag for links

> 📝 An element (e.g., `a href` attribute) is the complete structure, including the opening tag, content (if any), and the closing tag.

Table: HTML Tags

Tag	Description
`<html>`	The root element of an HTML document
`<head>`	Contains meta-information about the webpage
`<body>`	Contains the visible content of the webpage
`<h1>` to `<h6>`	Headings of various levels (`h1` being the largest)
`<p>`	Defines a paragraph
`<a>`	Defines a hyperlink
`<img>`	Embed an image
`<ul>`	Defines an unordered list
`<ol>`	Defines an ordered list
`<li>`	Defines a list item
`<table>`	Defines a table

Preparation for Web Scraping

Preparation

- Chrome browser
  - We assume you already have the Chrome browser installed on your machine!
- **Selenium**
- **Chrome driver**

Selenium

An open-source library that is designed for **automating web browsers**.

- Can browse web like a human
- Can be used to scraping information on web

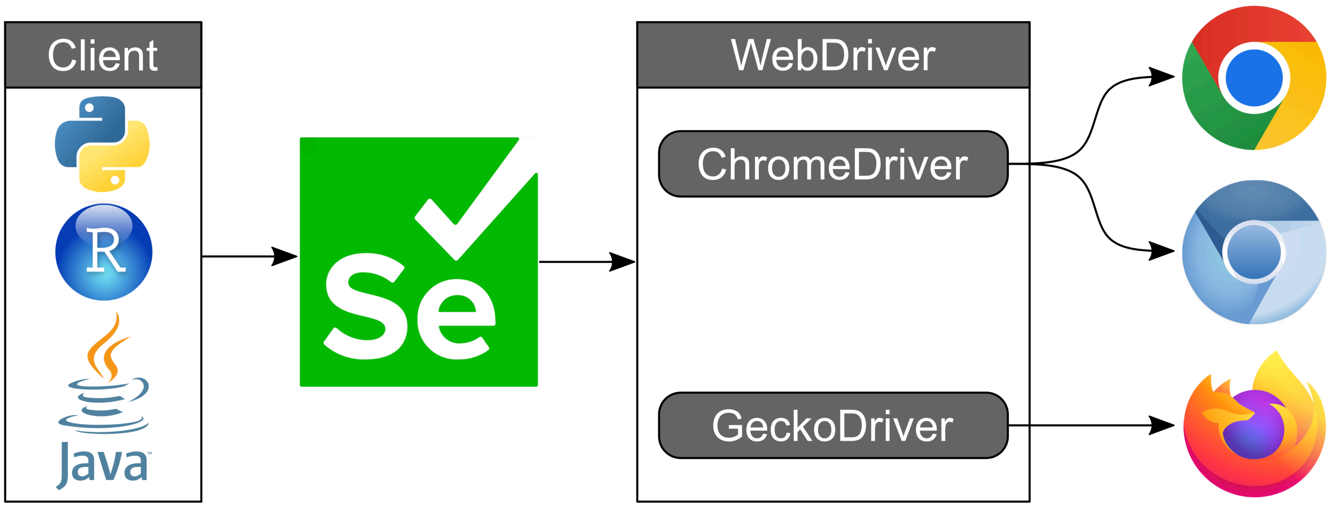
How to install:

1. Open the Anaconda prompt
2. Enter the following:

```
pip install selenium
```

Chrome Driver

A separate executable file that enables Selenium WebDriver to automate Google Chrome browsers.

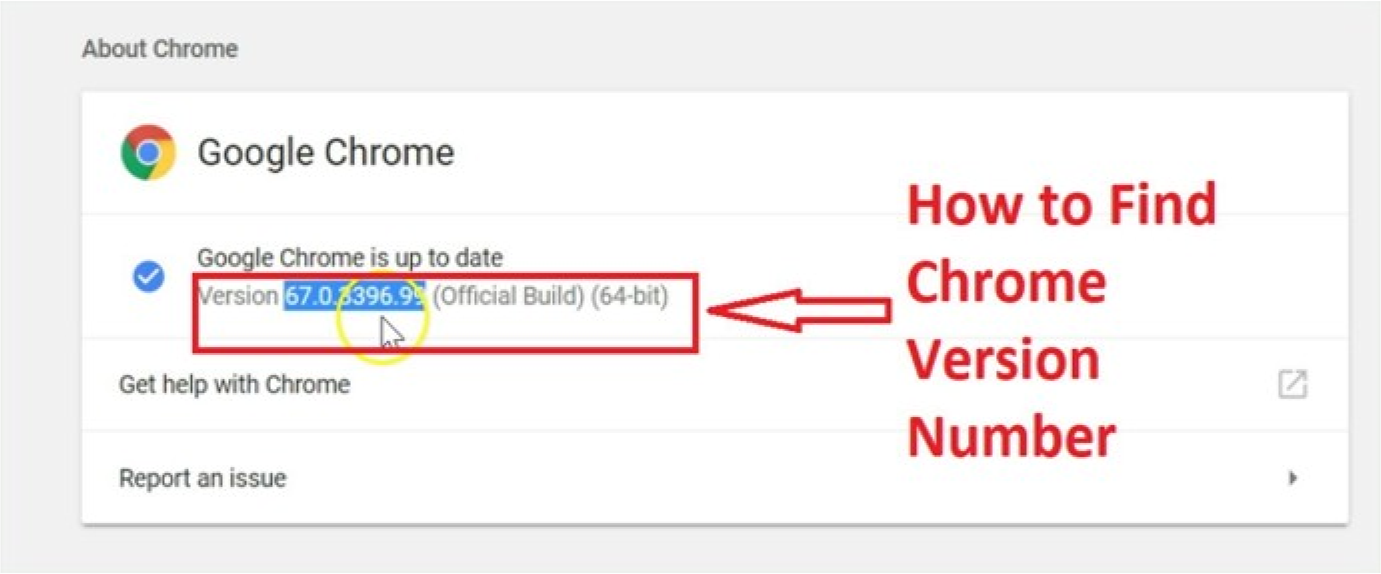


To use it, you need to **download a ChromeDriver version** that is **compatible with your Chrome browser version**.

How to Download Chrome Driver?

1. Update your Chrome browser to the latest version and check its version

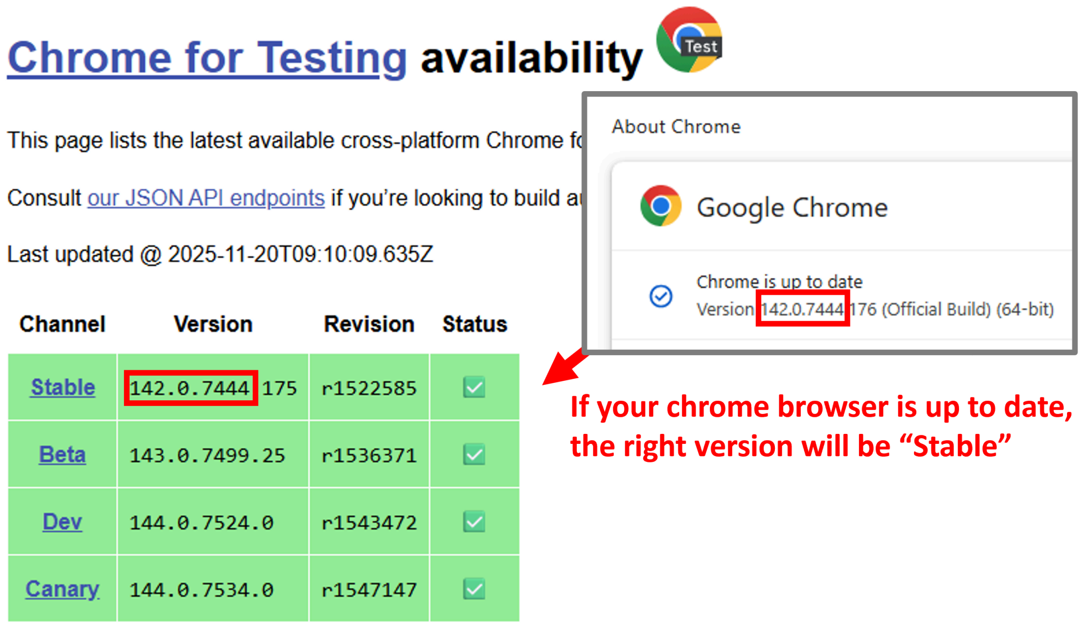
- Update your Chrome browser to the ***latest*** version (**Strongly recommended!**)
- You can check the version by entering the following url in the address bar: chrome://settings/help



2. Find the Correct Chrome Driver

- Go to the official Chrome for Testing availability dashboard: https://googlechromelabs.github.io/chrome-for-testing/
- Find the version that matches the one you just checked.

> 📝 If your Chrome browser is the **latest version**, the right version will be **Stable**.



Chrome for Testing availability

This page lists the latest available cross-platform Chrome fo

Consult our JSON API endpoints if you're looking to build a

Last updated @ 2025-11-20T09:10:09.635Z

Channel	Version	Revision	Status
Stable	142.0.7444.175	r1522585	✅
Beta	143.0.7499.25	r1536371	✅
Dev	144.0.7524.0	r1543472	✅
Canary	144.0.7534.0	r1547147	✅

> 📝 If your Chrome browser version is lower than those in the dashboard, update your Chrome or go to https://chromedriver.chromium.org/downloads.

3. Download the Correct Chrome Driver

- Download the chromedriver zip file according to your operating system (e.g., `win64` for Windows, `mac-x64` or `mac-arm64` for Mac).

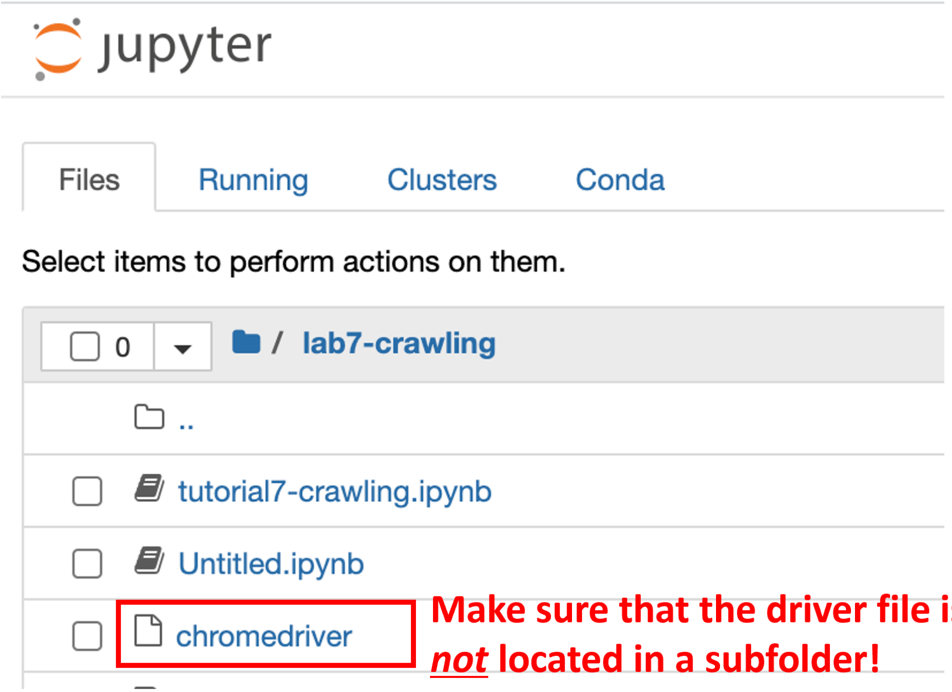
chromedriver	mac-arm64	Mac M1~	https://storage.googleapis.com/chrome-for-testing-public/142.0.7444.175/mac-arm64/chromedriver-mac-arm64.zip
chromedriver	mac-x64	Mac Before M1	https://storage.googleapis.com/chrome-for-testing-public/142.0.7444.175/mac-x64/chromedriver-mac-x64.zip
chromedriver	win32		https://storage.googleapis.com/chrome-for-testing-public/142.0.7444.175/win32/chromedriver-win32.zip
chromedriver	win64	Windows	https://storage.googleapis.com/chrome-for-testing-public/142.0.7444.175/win64/chromedriver-win64.zip

4. Unzip the zip file and you will get the driver file

- Windows: `chromedriver.exe`
- Mac: `chromedriver`

5. **Place the driver file in the same folder as the code file**

- e.g., the current Jupyter notebook file you are working on



Make sure that the driver file is *<u>not</u>* located in a subfolder!

Using Chrome Driver on Python

The code below will open a new selenium browser window:

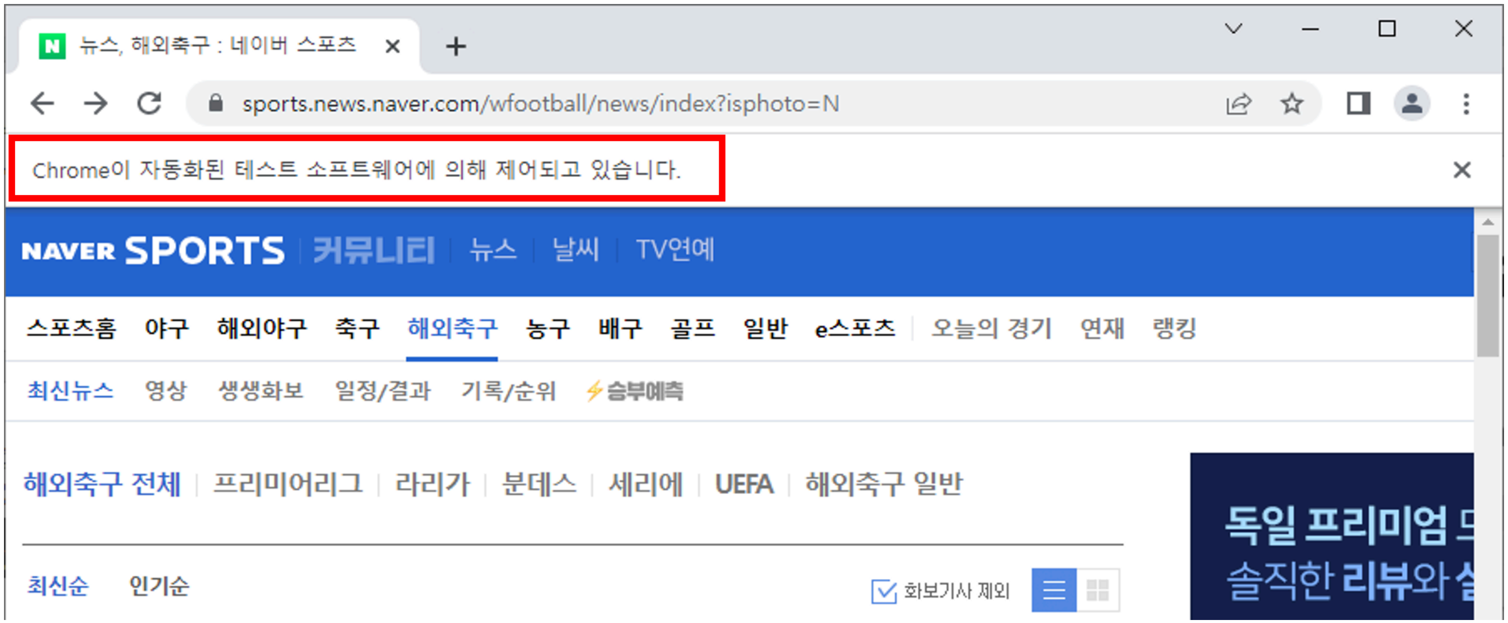
```
from selenium import webdriver

# set driver path depending on your OS
driver_path = './chromedriver.exe' # windows
driver_path = './chromedriver'     # mac

cService = webdriver.ChromeService(executable_path=driver_path)
driver = webdriver.Chrome(service=cService)
driver.implicitly_wait(3) # waiting 3 seconds
```

Selenium browser

Below the address bar, you will see the following text: `Chrome is being controlled by automated test software.`



> ⚠️ Don't close the opened browser until your web scraping process is completed!

> 📝 If your job is completed, run `driver.close()` to release `chromedriver` from memory and save your resource😄

[MacOS only] If you encounter the following error:

```
In [11]: from selenium import webdriver

         driver = webdriver.Chrome('/Users/ejk/Downloads/chromedriver')

/var/folders/78/m1bwm76j7yqcz3_s0cshbww80000gn/T/ipykernel_17379/3408759599.py:3: DeprecationWarning: executable_path
has been deprecated, please pass in a Service object
  driver = webdriver.Chrome('/Users/ejk/Downloads/chromedriver')

---------------------------------------------------------------------------
WebDriverException                          Traceback (most recent call last)
Input In [11], in <module>
      1 from selenium import webdriver
----> 3 driver = webdriver.Chrome('/Users/ejk/Downloads/chromedriver')

File /opt/homebrew/Caskroom/miniforge/base/envs/py39/lib/python3.9/site-packages/selenium/webdriver/chrome/webdriver.

    108 return_code = self.process.poll()
    109 if return_code:
--> 110     raise WebDriverException(
    111         'Service %s unexpectedly exited. Status code was: %s'
    112         % (self.path, return_code)
    113     )

WebDriverException: Message: Service /Users/ejk/Downloads/chromedriver unexpectedly exited. Status code was: -9
```

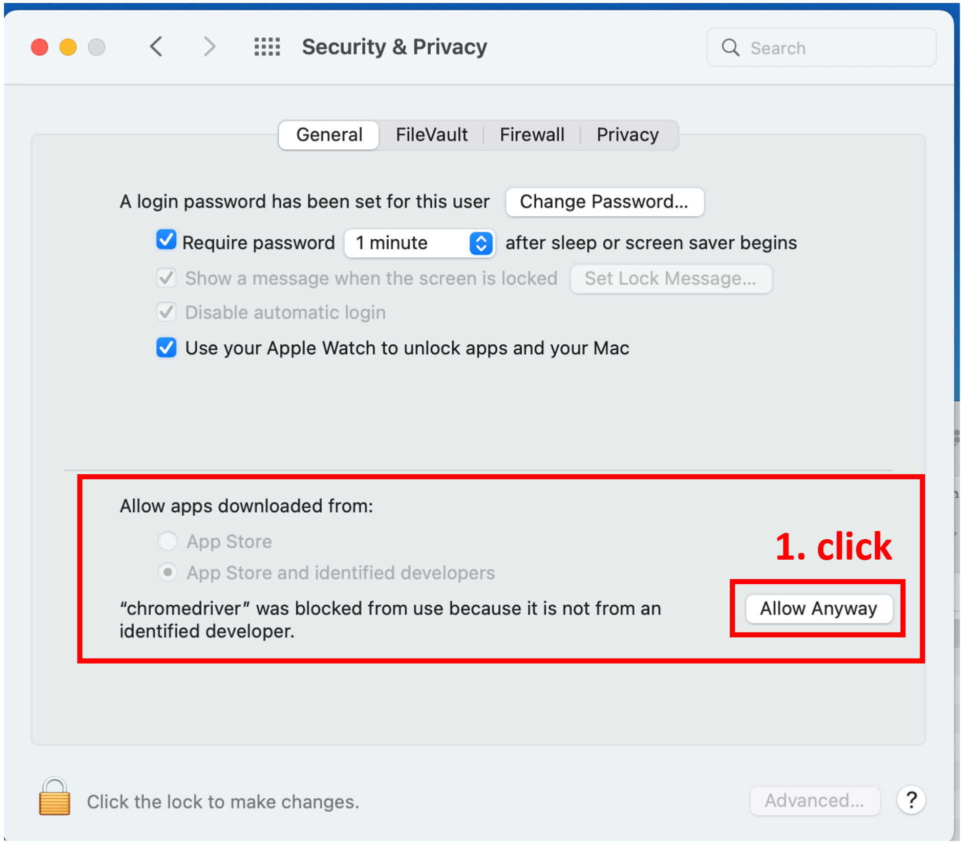
```
WebDriverException: Message: service /Users/... unexpectly exited. Status code was: -9
```

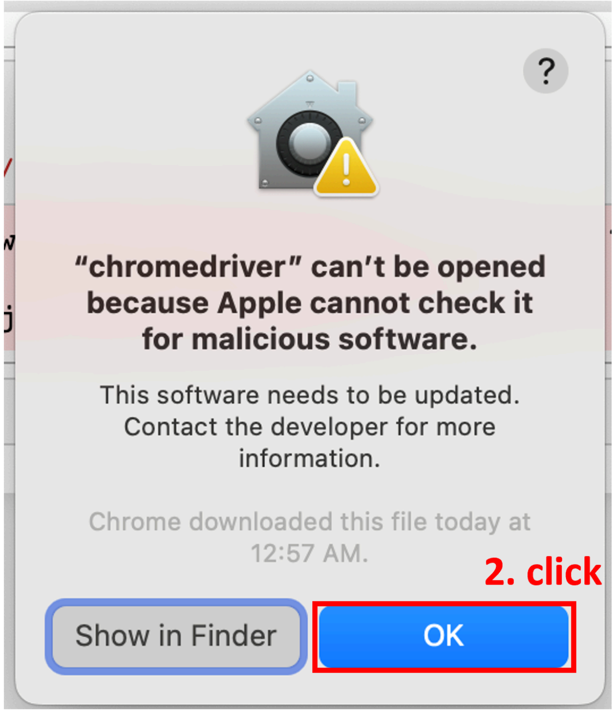
This is because MacOS prevented the Chrome driver from being opened.

Let's allow it to be opened!

How to fix it?

- `Security & Privacy setting` – `General tab`





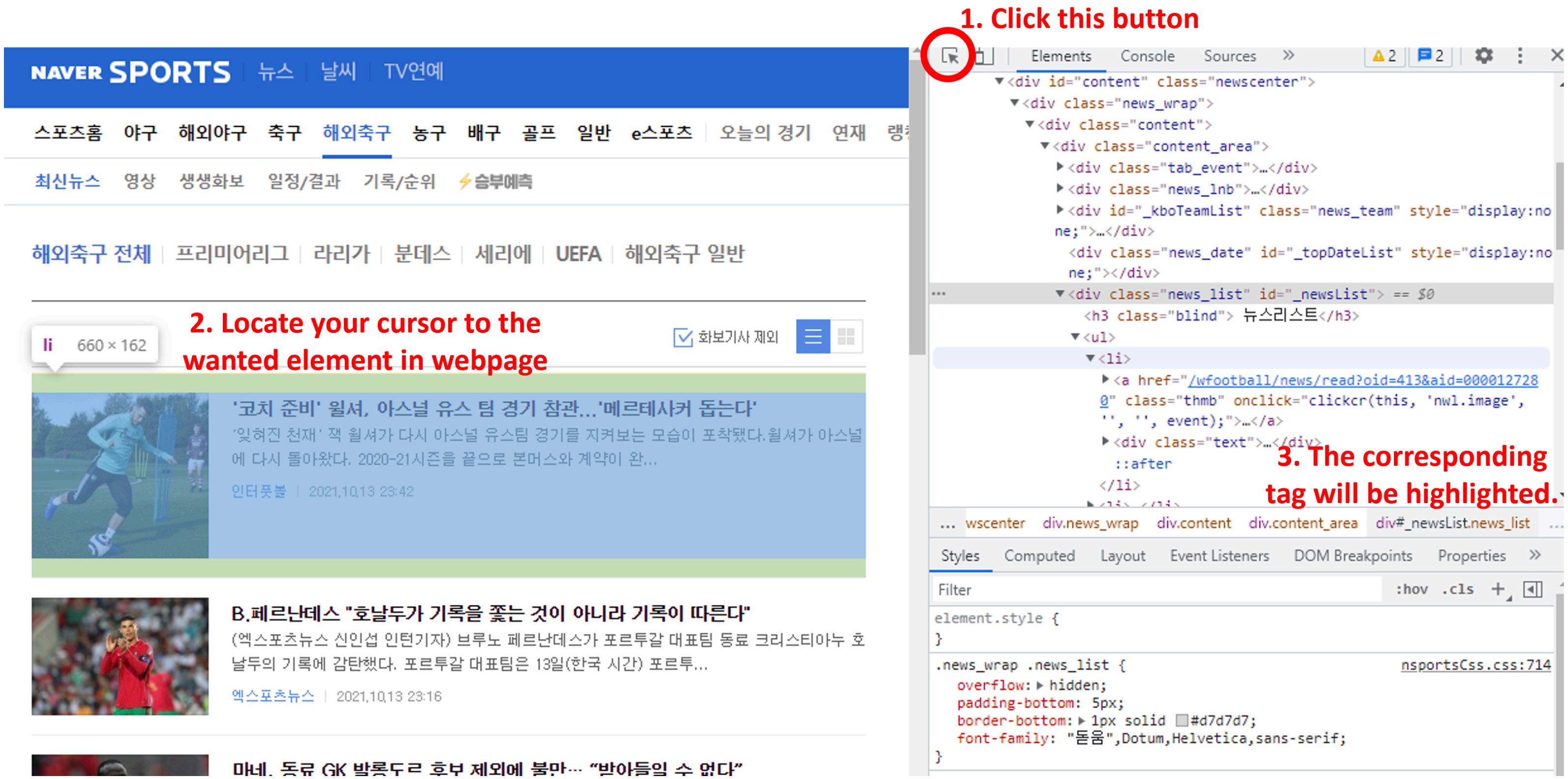
Web Scraping

Today's Menu: Daum Sports News

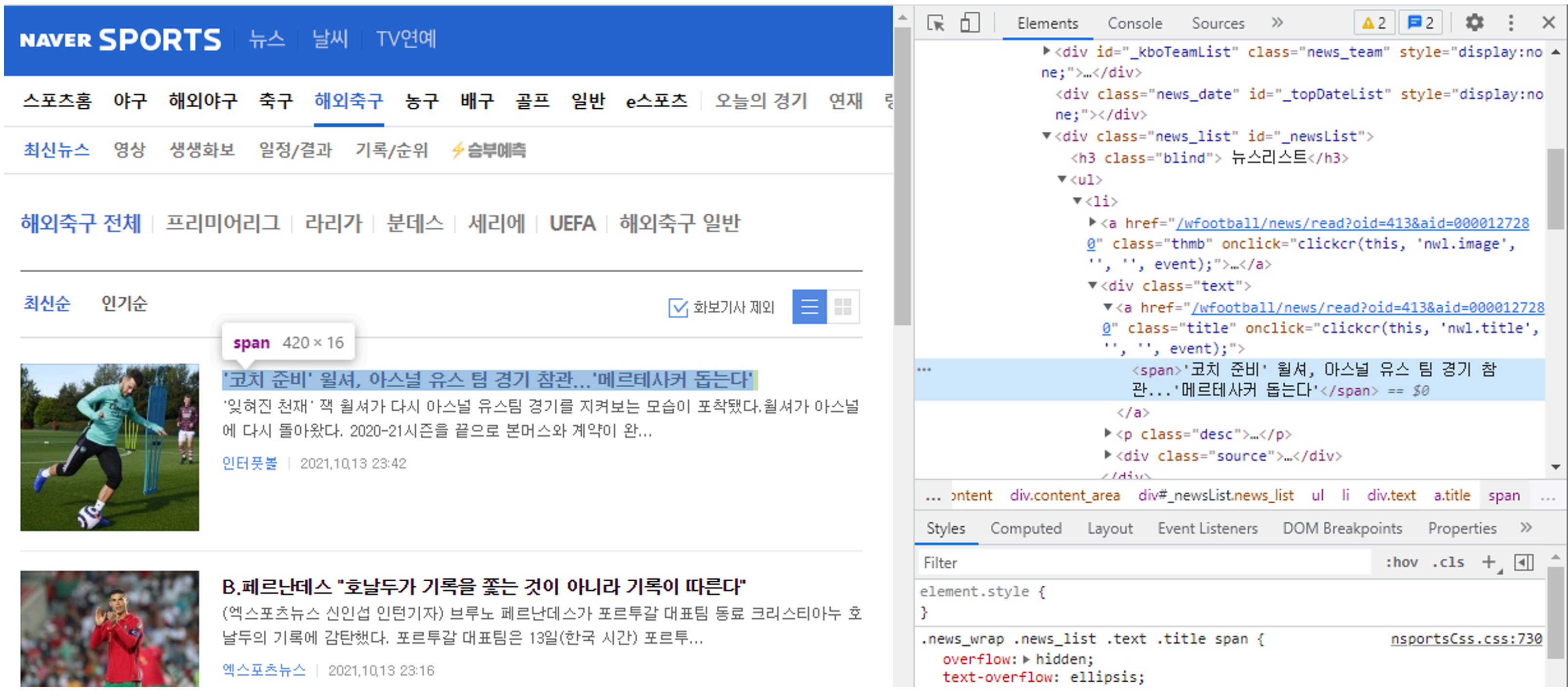
- Daum Sports News (https://sports.daum.net/worldsoccer/news/breaking)
- Scraping attributes of all news in the first page of the **'global football news (해외 축구)'** > **'news (뉴스)'** menu.
    - Title
    - News text
    - Publisher
    - Datetime

To scrap the data, you need to analyze the structure of the HTML code.

Finding the Wanted Tag



Moving cursor will change the location of the highlight!



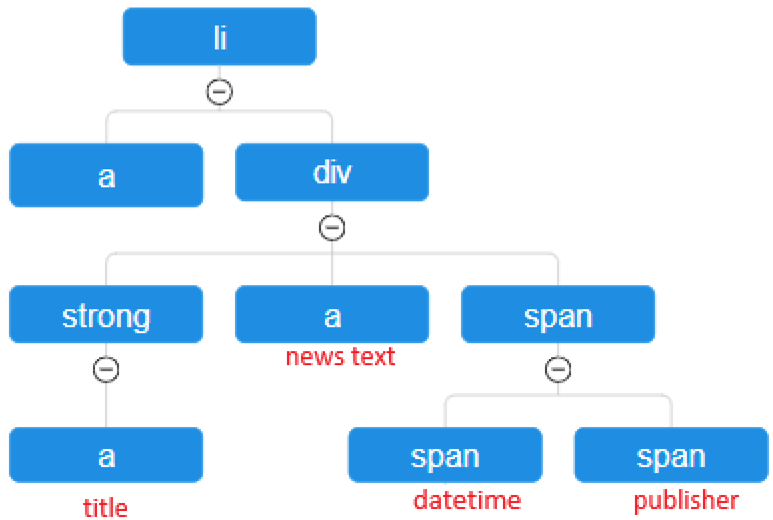
Findings

1. All news items in the page is contained in the `<ul class="list_news">` element. Each news item corresponds to a `<li>` element.

```
...
<ul class="list_news">
    <li data-searchid="313XXX"> ... </li>
    <li> ... </li>
    <li> ... </li>
    ...
    <li> ... </li>
</ul>
...
```

2. By analyze the tree structure of a `<li>` element, we can find the locations of target data:

```
<li data-searchid="313XXX">
    <a href="https://v.daum.net/v/BYnlkg5rhX"><img src="//img1...."></a>
    <div class="wrap_cont">
        <strong class="tit_news">
            <a href="..." class="link_txt" ...> 맨유 드디어...</a>
        </strong>
        <a href="..." class="link_desc"> [스타뉴스 | 박건도 기자] 유럽 소식에...</a>
        <span class="info_news">
            <span class="txt_info txt_num">
                <span class="screen_out">작성날짜</span>2분 전
            </span>
            <span class="txt_info">스타뉴스</span>
        </span>
    </div>
</li>
```

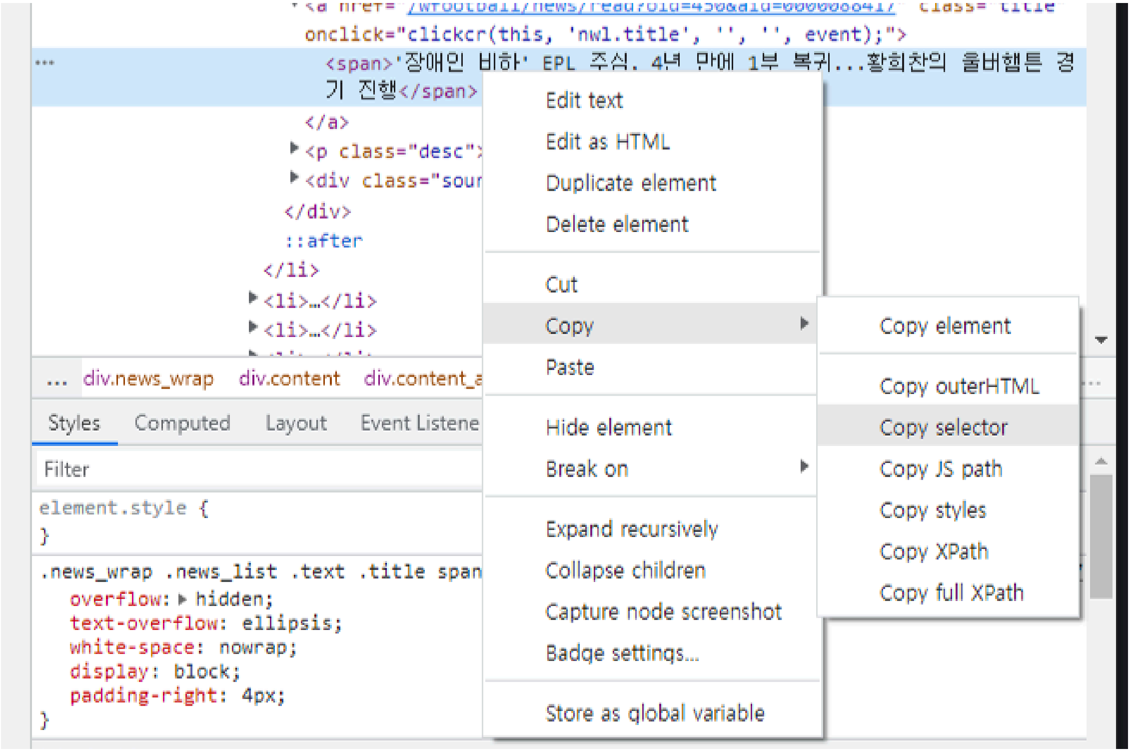


- Title: `li > div > strong > a`
- News text: `li > div > a`
- Publisher: `li > div > span > span`
- Datetime: `li > div > span > span:nth-child(2)`

> 📝 These patterns are called **CSS Selector**!
> A CSS selector is a pattern used to "find" or select HTML elements on a webpage

CSS Selector in Developer Tool in Chrome

You can copy the CSS selector pattern of the target element:



Homework: Data Scraping

Naver Entertainment Photo News page (https://m.entertain.naver.com/photo) contains photos of famous idols and actors.

1. Scrap the image urls and titles of all photo items in the URL.

In HTML, an image is displayed using a `<img>` tag element:

`<img src="https://..." alt="[MD포토]...">`

- image url: `src` attribute value (`https://...`) of `<img>` tag
  - If you enter the url into the browser address bar, a picture should appear.
- title: `alt` attribute value of `<img>` tag

2. Save the data in CSV format.

- Column names: `title`, `url`
- File name: `entertain.csv`

> 💡 You can get the value of an element's HTML attribute using `get_attribute()`.

```
element.get_attribute("attribute_name")
```

For example, let the Python variable name of an `<img>` tag element be `x`. Then, you can get `src` attribute value using `x.get_attribute("src")`.

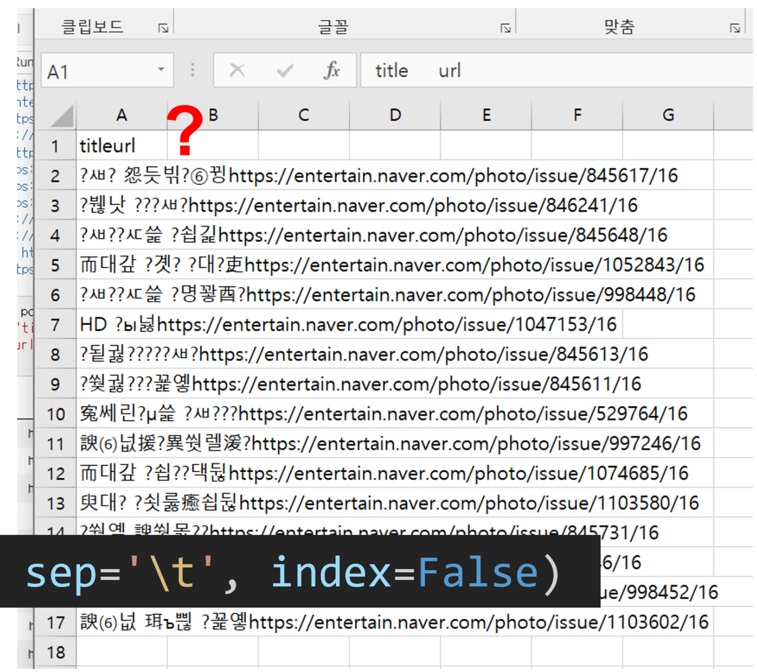
FAQ) What's wrong with my file?

I crawled the data and saved it to a `csv` file (left). But, when I opened the `csv` file using MS Excel, there are ***unreadable*** characters (right) that differ from what I see in Python.

```
In [21]: import pandas as pd
         df=pd.DataFrame({'title':title_list,
                          'url':url_list,})
         df
```

Out[21]:

	title	url
0	스타 공항패션	https://entertain.naver.com/photo/issue/845617/16
1	화보 속 스타	https://entertain.naver.com/photo/issue/846241/16
2	스타들의 일상	https://entertain.naver.com/photo/issue/845648/16
3	컴백 티저 이미지	https://entertain.naver.com/photo/issue/105284...
4	스타들의 인터뷰	https://entertain.naver.com/photo/issue/998448/16
5	HD 포토	https://entertain.naver.com/photo/issue/104715...
6	행사장 속 스타	https://entertain.naver.com/photo/issue/845613/16
7	시사회 현장	https://entertain.naver.com/photo/issue/845611/16
8	경기장의 스타들	https://entertain.naver.com/photo/issue/529764/16
9	방송국 출근길	https://entertain.naver.com/photo/issue/997246/16
10	컴백 쇼케이스	https://entertain.naver.com/photo/issue/107468...
11	[illegible]	[illegible]
12	[illegible]	[illegible]
13	[illegible]	[illegible]
14	뮤지컬·연극 무대	https://entertain.naver.com/photo/issue/998452/16
15	방송 촬영 현장	https://entertain.naver.com/photo/issue/110360...




```
df.to_csv('entertain.csv', sep='\t', index=False)
```

A. Short answer

If you are planning to open the csv data using Excel, set the encoding of csv file to `cp949`!

```
df.to_csv('entertain.csv', sep='\t', encoding='cp949')
```

To reload the `cp949` csv file in Python without any broken characters, just set the encoding of the csv file!

```
df = pd.read_csv('entertain.csv', sep='\t', encoding='cp949')
```

Further Explanation

The problematic situation is happend because MS excel opened the csv file in `utf-8` using `cp949`.

By default, Python uses `utf-8` encoding.

- This encoding supports many languages and can accommodate strings in any mixture of those languages.
- Thus, if you do not set any encoding, the data will be saved in `utf-8` encoding.

```
# Both lines of code do the same job: saving the data with `utf-8` encoding.
df.to_csv('data.csv')
df.to_csv('data.csv', encoding='utf-8')
```

Meanwhile, MS Excel uses `cp949` encoding.

- Microsoft has built this Korean-specific encoding to cover numerous Hangul syllables.

Thank You.